

¿QUÉ TAN LEJOS VIAJA UN DETECTOR DE BASURA?

DETECCIÓN CROSS-DOMAIN DE RESIDUOS URBANOS EN IMÁGENES DE MANO, DASHCAM Y DRON

Jairzinho Santos

Universidad Nacional de Ingeniería, Lima, Perú — Visión Computacional, Maestría en
Inteligencia Artificial, 2026-I — Prof. Elian Laura Riveros

La pregunta

Los benchmarks de detección de basura se evalúan **dentro de su dominio**, pero una ciudad que despliega sensores mixtos — fotos ciudadanas, dashcams de camiones, drones — asume implícitamente que los detectores transfieren entre puntos de vista. **Nadie había medido si lo hacen.** Entrenamos y evaluamos en tres dominios públicos bajo un protocolo auditado:

Dataset	Vista	Imágenes	Obj. mediano	% small
TACO	mano	1,500	171 px	25
RoLID-11K	dashcam	11,564	22 px	84
UAVVaste	dron	772	72 px	66

19 experimentos, 4 familias de modelos (Faster R-CNN, YOLOv11n, FCN, VGG+CAM); toda métrica sale del mismo contrato pycocotools contra ground truths congelados — tablas y figuras se generan, nunca se transcriben.

82%

caída cross-domain de mAP₅₀

+21%

inflación de AP₅₀ por fuga

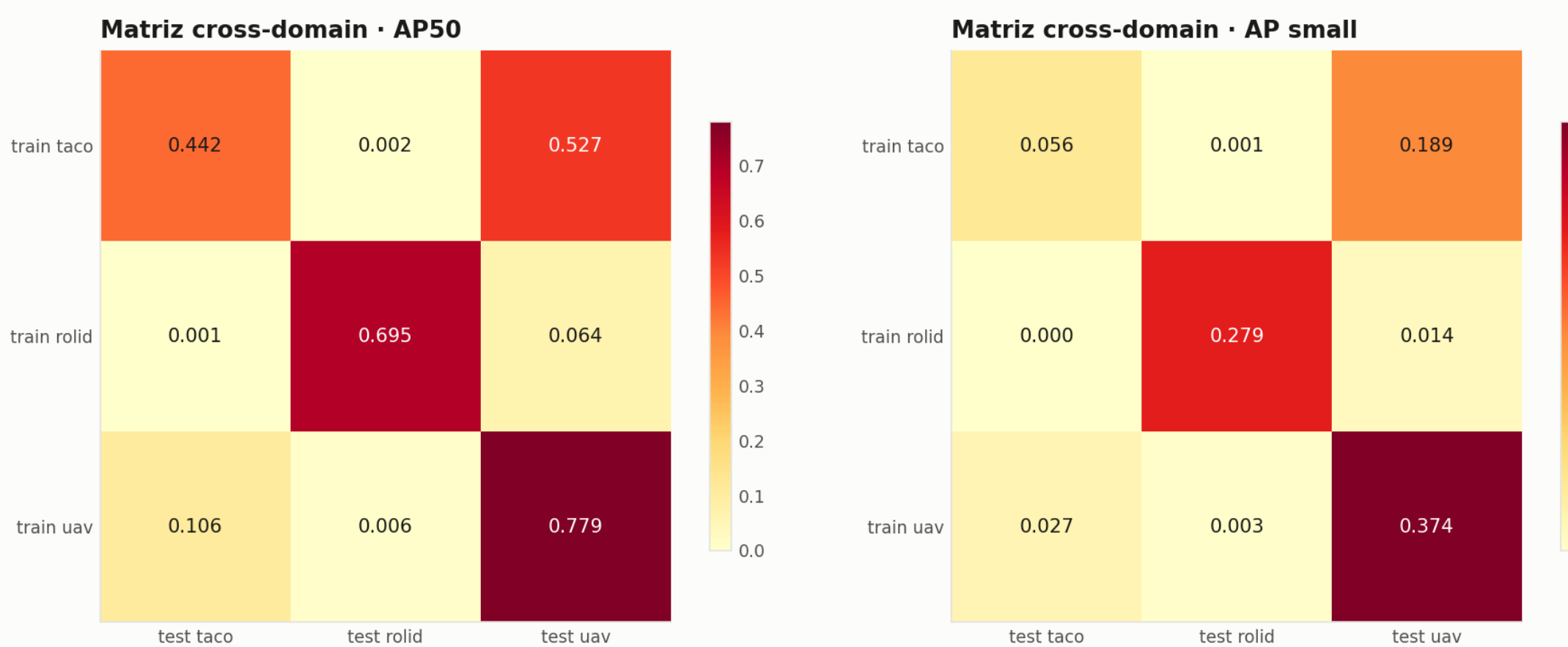
+8.3

AP₅₀ con 640→1024 px (dashcam)

16×

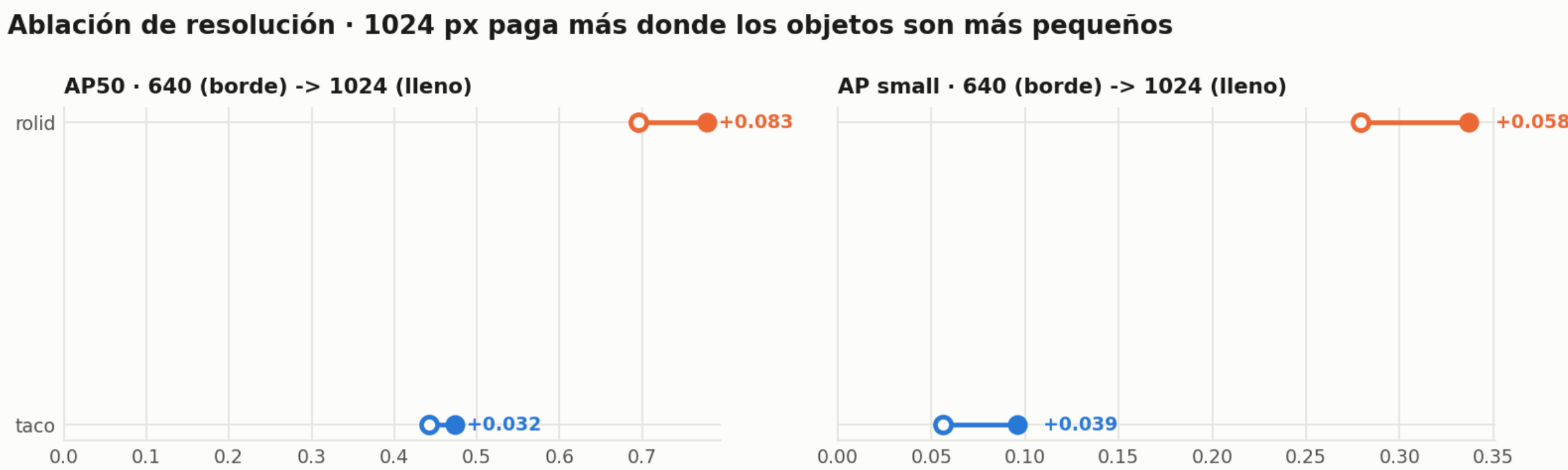
menos parámetros, a 1–4 puntos

Los detectores no viajan



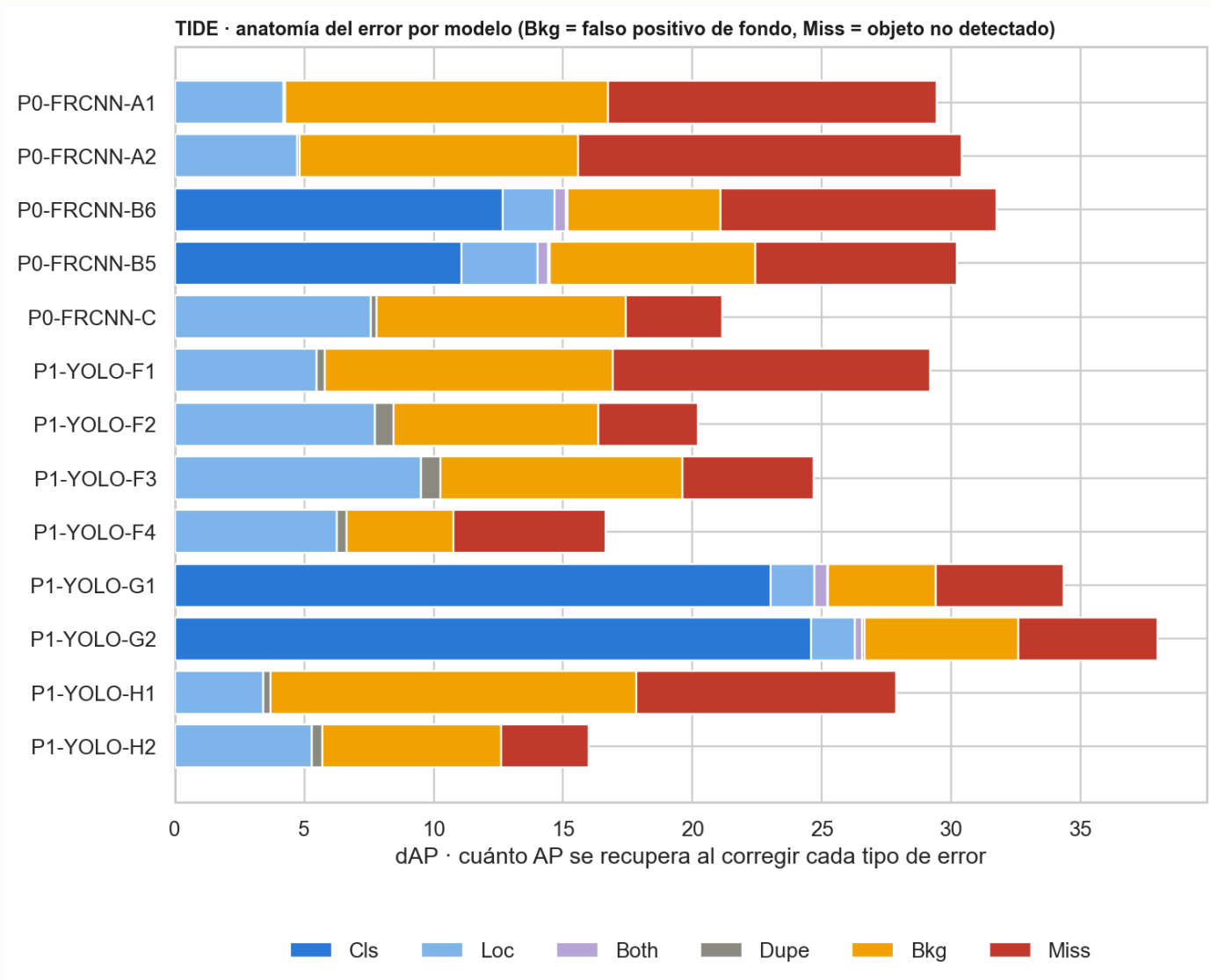
Media in-domain de 0.639 AP₅₀; cruzada, 0.118. El dominio dashcam es una **isla** (AP₅₀ ≤ 0.064 en ambas direcciones): su objeto mediano de 22 px cae bajo el ancla FPN mínima de 32 px. Mano→dron retiene el 68% — la escala, no la apariencia, domina la brecha.

Palancas baratas: la resolución



Las ganancias se concentran donde los objetos son más pequeños: dashcam +8.3 AP₅₀ / +5.8 AP_S; mano +3.2 / +3.9. Granularidad: excluir el vidrio marginal (38 instancias de test, AP₅₀ 0.049) sube las cinco clases restantes +0.014 AP₅₀ en promedio.

Por qué fallan: anatomía del error con TIDE



Los multiclase pierden hasta **24.6 dAP por clasificación** contra 1.7 por localización: el cuello de botella es nombrar el material, no encontrar la basura; los dashcam in-domain casi no fallan por Miss.

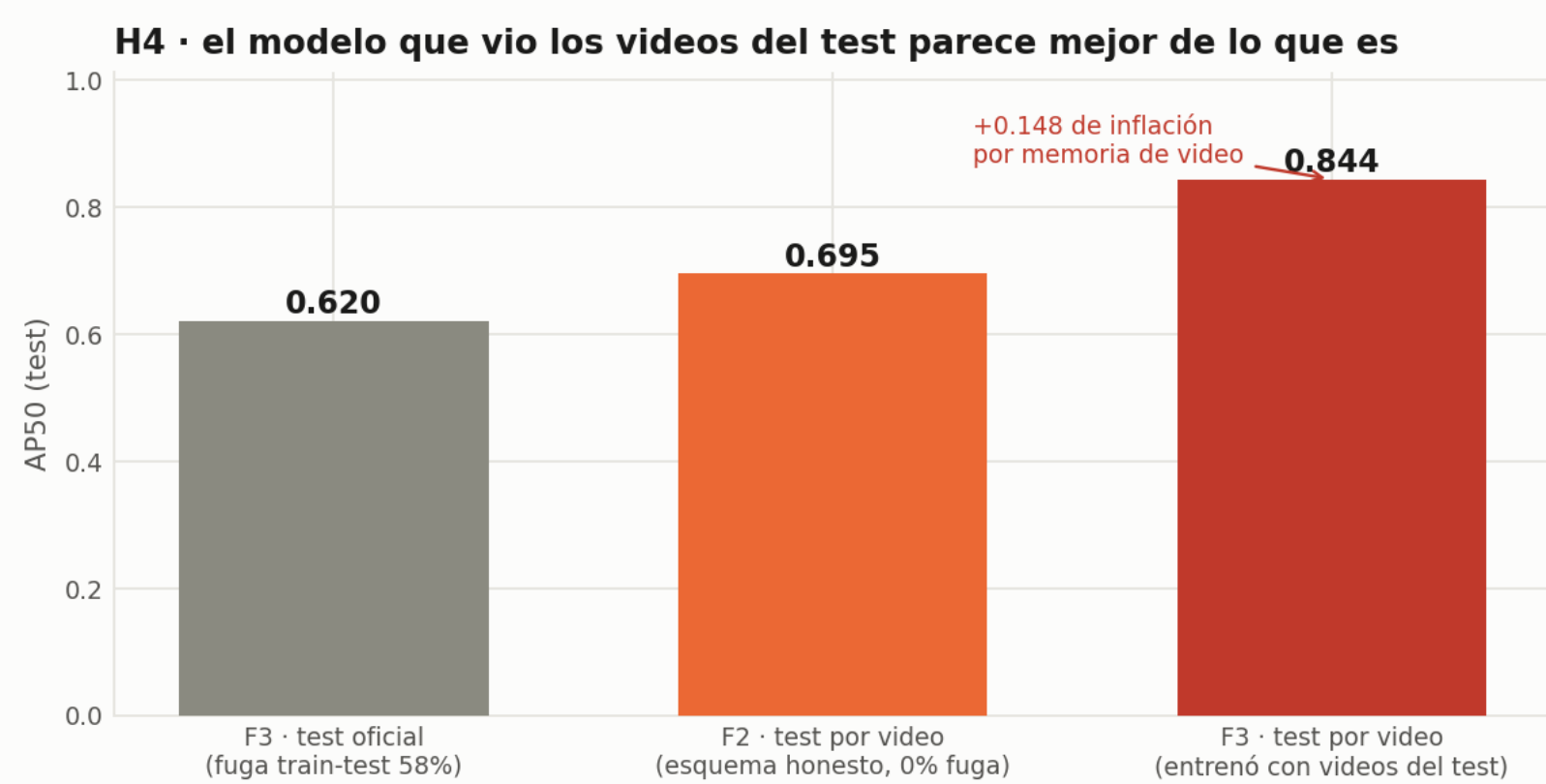
Auditar antes de entrenar: 7 defectos hallados

Verificaciones sistemáticas (hashing perceptual, verificación EXIF, cruce de splits) sobre los datasets publicados destaparon:

- **H4 — fuga a nivel de video en los splits oficiales de RoLID-11K:** el 58.2% del test comparte video de origen con el entrenamiento (87% de redundancia visual intra-video). **Los baselines publicados están inflados por memorización de escena.**
- H1/H2 — colisiones de nombres en TACO entre lotes; ~37% de rotaciones EXIF pendientes con cajas anotadas sobre el marco rotado.
- H5/H6 — imagen duplicada entre val y test (RoLID); par casi duplicado cruzando train/test (UAVVaste).
- H3/H7 — las carpetas de la distribución no codifican la hora; marcos de anotación inconsistentes entre datasets.

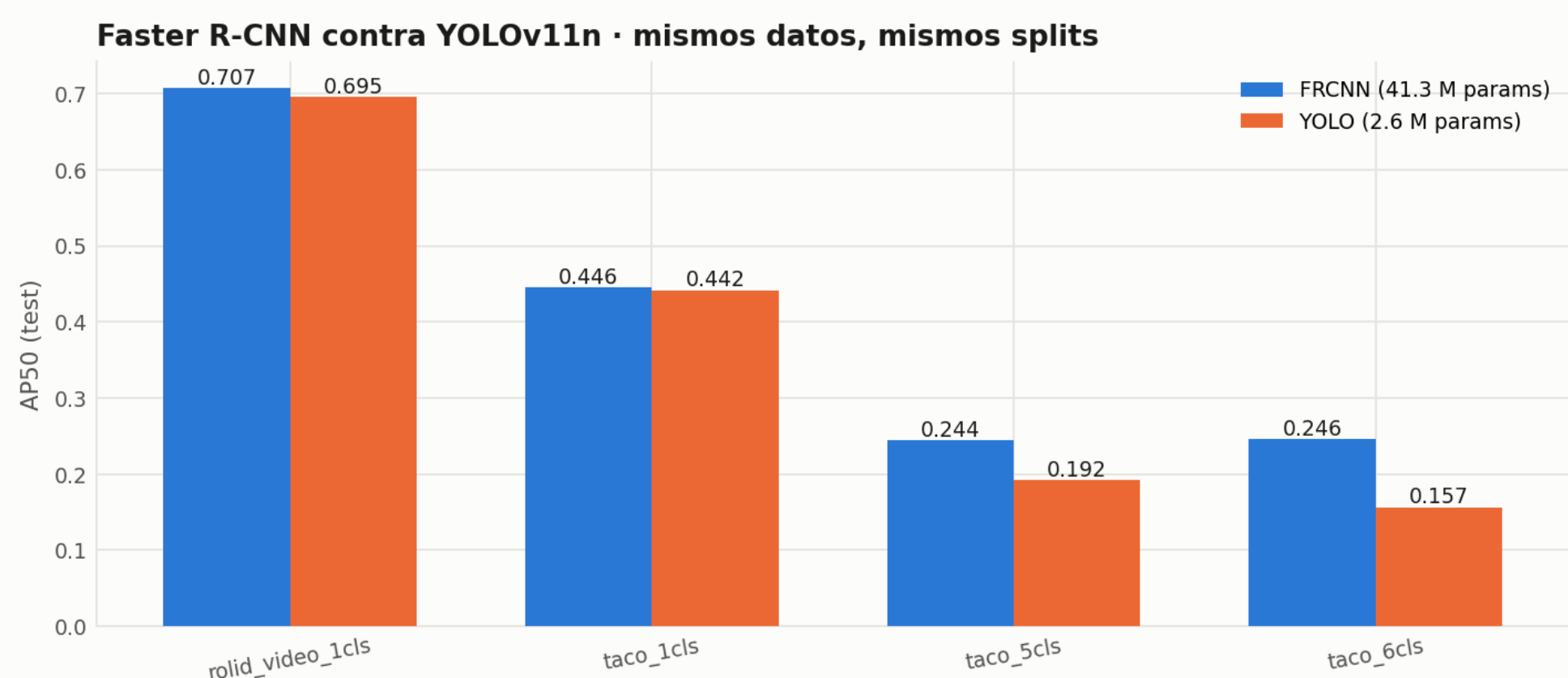
La cura: pipeline medallón (raw→bronze→silver →gold) con linaje SHA-256, cinco correcciones y splits congelados conscientes de grupo (grupos pHash / video de origen; la estratificación iterativa deja cada clase a 0.1 pp del objetivo de 15% de test).

La higiene del split cambia conclusiones



Sobre el *mismo* test limpio, el modelo entrenado con el split oficial fugado marca **0.844 contra 0.695** del entrenado limpio: **+21% relativo** — escenas memorizadas, no generalización. El split por video lo elimina a costo cero.

41M contra 2.6M de parámetros, mismos splits



YOLOv11n queda a 1–4 puntos de AP₅₀ de Faster R-CNN en basura de una clase (0.695 contra 0.707 en dashcam) — el despliegue ligero es viable. La brecha se abre en materiales (0.157–0.192 contra 0.244–0.246). FCN: 0.577 de IoU; recortes VGG-16: 97.8% de exactitud con evidencia CAM sobre el objeto.

Conclusiones y reproducibilidad

- **Aún no hay modelo universal de basura:** el ajuste por dominio (o al menos adaptar la resolución) sigue siendo obligatorio con sensores mixtos.
- **Auditar antes de entrenar:** una fuga escondida reescribe un benchmark; los datasets con estructura de video deberían publicar identificadores de grupo.
- **Detectar agnóstico a la clase y clasificar recortes:** TIDE aboga por desacoplar el material (97.8% en recortes) de la detección.

Tres rutas publicadas: demo con pesos preentrenados sobre cualquier foto (~5 min, CPU) · regenerar cada número del paper desde los artefactos (~10 min) · pipeline completo desde datos públicos. Código, pesos, splits y video: github.com — **se publica con la entrega del curso.**